

Subagents vs Agent Skills: Executing Reusable Knowledge for Long-Horizon Agentic Tasks

Wasu Top Piriyakulkij^{*1} Rachel Lawrence² Alicia Curth²
 Sushrut Karmalkar² Niranjani Prasad²
 Cornell University¹ Microsoft Research Cambridge²

Abstract

How can language model agents effectively leverage libraries of reusable knowledge to solve long-horizon tasks? Recent work has increasingly focused on agent skills: reusable capabilities represented as skill packages, i.e., multi-file bundles containing instructions, scripts, and other resources that help agents perform specific tasks. Agent skills are typically executed by loading their skill instructions into an agent’s context and relying on the agent to follow them. As task horizons grow, however, this approach becomes increasingly brittle, because reasoning quality degrades as more information accumulates in the context window. We investigate an alternative approach in which skill packages are instead invoked as subagents. Rather than loading skill instructions into the main context, subagent execution spawns fresh context windows dedicated to solving individual subtasks. We show that subagent execution outperforms agent-skill execution when skill packages expose clear input-output contracts and their instructions encode the procedural knowledge needed to fulfill those contracts. The tradeoff is additional communication overhead, as extra tokens are required to coordinate between the main agent and its subagents. Our results show that the benefit of reusable knowledge depends not only on its content, but also on how it is organized and invoked.

1 Introduction

How do we equip AI agents with domain-specific knowledge so that they can solve complex tasks? This is a longstanding question in AI, dating back to the symbolic AI era, where researchers attempted to encode knowledge explicitly through structured representations such as rules, logic, and knowledge graphs [1, 2, 3, 4]. The rise of deep learning shifted the paradigm from direct knowledge injection to weight-based learning; knowledge is acquired from data and stored implicitly in neural weights. The advent of large language models (LLMs) has partially shifted the field back toward explicit knowledge injection. Through in-context learning, LLMs acquire and apply new information at inference time, without weight updates, in one of the most expressive symbolic representations: natural language.

Recently, *agent skills* [5] have emerged as an effective way of knowledge injection for LLM agents. Each agent skill is represented by a *skill package*: a multi-file package containing instructions and scripts that are useful for solving specific tasks. The names and descriptions of available skills are then given to the LLM agent. When the agent invokes a skill, it gains access to that skill’s instructions and follows them. These agent skills have become the dominant way of providing LLM agents with reusable, domain-specific knowledge.

Despite their name, agent skills are quite different from the notion of a skill in reinforcement learning [6, 7, 8]: a skill is a temporally extended policy invoked by a higher-level controller. Invoking agent

^{*}Work done during an internship at Microsoft Research Cambridge.

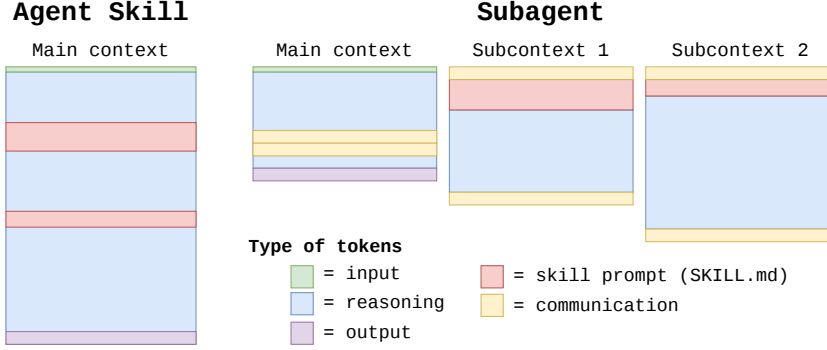


Figure 1: Two type of skill package execution: agent skill vs subagent. While agent-skill execution does all the reasoning in a single context, subagent execution creates new context windows for each subtasks, each window initialized with skill instructions (SKILL.md). Since subagents reason within their own context, however, extra tokens are needed for the main agent and subagents to communicate.

skills, instead, simply loads the skill recipes (SKILL.md files) into the LLM agent’s context, rather than directly performing the skill or solving a subtask. Thus, even when an agent skill contains relevant information, invoking it might be unproductive, as adding more instructions increases context length, and agent performance degrades as context grows [9, 10, 11, 12]. This issue of bloated context becomes increasingly problematic as task horizons become longer.

We therefore investigate an alternative approach to agent skills: executing skill packages as *subagents*. Instead of loading skill instructions into the main context, subagent execution spawns a new context window seeded with the skill instructions, performs the skill independently, and returns only the output to the main agent. We possess that by dividing a task across multiple context windows, the maximum amount of information processed by any individual context can be reduced (Figure 1). While existing agent harnesses do have subagents, they are generally underutilized, targeting mainly situations where independent subtasks can be run in parallel to reduce latency [13]. Executing skills as subagents has downsides, however. For instance, we lose the ability to combine knowledge from multiple skills to solve a subtask.

In this work, we show that a skill package is suitable for subagent execution when it is presented as procedural knowledge with clear input and output contracts, analogous to the options framework in reinforcement learning[6]. Procedural skills with well-defined input-output interfaces are inherently self-contained, allowing their internal reasoning to be delegated to separate contexts. On SkillsBench, a benchmark for evaluating agent skills, we synthesize procedural, contract-driven skill packages derived from successful task trajectories and show that executing these skill packages as subagents significantly outperforms agent-skill execution. Our findings suggest that both how skills are executed and how skill knowledge is organized are important design choices that substantially impact agent performance.

2 Background: Agents with Tools, Agent Skills, and Subagents

We are interested in a tool-calling language model agent that operates iteratively. Let c_t denote the agent’s context at turn t . For notational simplicity, we assume the agent invokes one tool per turn. At each turn t , the language model π_{LLM} generates a text response r_t , a tool k_t , and tool arguments x_t :

$$(r_t, k_t, x_t) = \pi_{LLM}(c_t) \quad (1)$$

The selected tool is executed by a tool executor E :

$$o_t = E(k_t, x_t) \quad (2)$$

where o_t denotes the tool output. The agent context is then updated by appending the response, tool call, and tool result:

$$c_{t+1} = c_t \oplus (r_t, k_t, x_t, o_t) \quad (3)$$

We slightly abuse notation and use $\oplus$ to denote appending an interaction trace to the context. Any structured objects are assumed to be converted into their textual representations before concatenation. The agent repeats this process until a final answer is produced at some turn T .

Agent Skills. An agent skill is specified by a skill package s :

$$s = (d, m, R) \quad (4)$$

where the description d provides a concise summary of the skill’s purpose and is always given to the main agent for skill discovery, the instruction file m (implemented as `SKILL.md`) specifies how the skill can be performed, and the resource collection R contains supporting artifacts such as scripts, examples, reference documents, and nested directories.

An agent skill is a tool constructed from a skill package, $k = \text{AgentSkill}(s)$, and we write m_k for the instruction file of its underlying package. Invoking an agent skill requires no additional arguments and simply returns the skill instructions m_k :

$$E(k = \text{AgentSkill}(s), \emptyset) = m_k. \quad (5)$$

The skill contents are thus exposed directly to the main context, and the procedure prescribed by m_k is carried out by π_{LLM} itself: each step of the skill’s execution is an ordinary agent step in the main context c , applying Equations (1) to (3)

Subagents. Subagents provide an alternative mechanism for utilizing reusable knowledge. Like agent skills, a subagent is constructed from a skill package, $k = \text{Subagent}(s)$. Unlike agent skills, however, invoking a subagent does not expose the skill instruction file directly to the main agent, nor is its execution carried out by π_{LLM} . Instead, the tool executor instantiates a distinct policy that conditions on the skill instructions m_k and initializes a new agent context from the task input x_t :

$$\pi^{(k)}(\cdot) \triangleq \pi_{LLM}(m_k \oplus \cdot), \quad c_0^{(k)} = x_t. \quad (6)$$

The subagent then executes an independent tool-calling reasoning process within this separate context, following Equations (1) to (3) under $\pi^{(k)}$, and returns only the response $r_T^{(k)}$ from its last turn T :

$$E(k = \text{Subagent}(s), x_t) = r_T^{(k)}. \quad (7)$$

Both mechanisms draw on the same skill package s , but they differ in which policy that knowledge conditions. Agent skills let a single policy π_{LLM} operating over a single context, with the skill’s execution unfolding as part of the main agent’s own turn sequence; subagents introduce a separate subpolicy $\pi^{(k)}$ per skill package, each acting on a context the main agent neither reads nor writes. Note that $\pi^{(k)}$ is defined here over the same LLM as the main π_{LLM} , but it can instead be instantiated from a different one; what makes it a separate policy is that it is separately instantiated and communicates with the main agent only through the input and output, x_t and $r_T^{(k)}$.

3 Subagents For Long-horizon Agentic Tasks

LLM reasoning capability declines with context length [9, 10, 11, 12]; this decline has been linked to bandwidth-limited attention: LLMs can only communicate a bounded amount of information across long inputs [14]. Long-horizon agentic tasks are particularly susceptible to this degradation: they require agents to reason over increasingly long trajectories, consisting of verbose tool inputs and outputs, and intermediate reasoning. While skill packages may contain useful knowledge to solve such tasks, invoking an agent skill, which loads the skill instructions into the agent’s context, amplifies the context overload issue. Solving long-horizon tasks may depend not only on having access to useful skills, but also on executing those skills in a way that controls context growth. To achieve this, we explore the use of *subagents*.

3.1 Reducing Peak Context Length Through Subagent Execution

To control context growth, we directly ask whether spawning more context windows can help. What matters for bandwidth limitations discussed above is not total context consumed across a task, but the *peak* context length any single window must process. If we can decompose a task into smaller, self-contained subtasks and execute each subtask within a separate context window, we expect the peak context length across all these windows to be shorter than that of a single, monolithic context window. Subagent execution encodes this exact mechanism (Figure 1). When a subagent is invoked,

a new context window is spawned and initialized with the subagent’s input and its skill package’s instructions.

One way to view subagent execution is through the lens of information encapsulation. Subagents encapsulate subtask-specific information. Subagents’ internal trajectories are hidden away from the main, parent agent. Only the final output of the subagent is visible to the main agent. The main agent then has less information to reason over, and so it can reason better.

However, peak context reduction with subagents comes at a cost. Executing a task across multiple context windows introduces communication overhead between the main agent and subagents (Figure 1): since subagents do not see the main context, they must be given sufficient input information to carry out their subtask, which means relevant information is often repeated across context windows. Subagent execution thus trades a higher total token count for reduced peak context length.

We note that popular agent harnesses such as Claude Code and OpenAI Codex do support spawning subagents, but they tend to be used for task parallelization rather than encapsulating information [13]. In many cases, the spawned subagents do not use any skill package; they function as lightweight parallel workers. The use of subagents as a mechanism for executing reusable knowledge remains relatively underexplored.

3.2 Input-Output Contracts for Effective Subagents

Reducing peak context length with subagents is useful for agent performance *only if* the subtasks in the subcontext windows are actually solved correctly. Unlike agent skills, subagents introduce an additional subtask delegation problem: the main agent must identify an appropriate subagent for a subtask and provide sufficient information for the subagent to solve it successfully. If the main agent fails at either of these, subagents become ineffective.

To ensure the main agent correctly delegates subtasks and provides required information for each subtask, each skill description needs to tell the main agent what subtask the corresponding subagent can solve and what information should be supplied to it. Of course, the skill instructions then must match the skill description: they have to tell the subagents how to solve the subtasks the description promises to solve.

Skill packages designed this way echo the options framework [6]: invoked as subagents, they behave as language-based options. An option is defined as

$$\omega = (\mathcal{I}_\omega, \pi_\omega, \beta_\omega) \quad (8)$$

where $\mathcal{I}_\omega$ is the initiation set, π_ω is the option policy, and β_ω is the termination condition. An option may be invoked only in states belonging to $\mathcal{I}_\omega$, after which it follows π_ω until termination occurs according to β_ω .

Similarly, an effective subagent skill package should specify when the subagent can be appropriately invoked, how it solves the delegated task, and what information it returns upon completion. We therefore structure the skill description as

$$d = (q_{\text{in}}, h, q_{\text{out}}) \quad (9)$$

where h is a concise summary of the skill’s purpose, and q_{in} and q_{out} are natural-language input and output contract specifications, respectively.

Substituting into Equation (4), the skill package is

$$s = ((q_{\text{in}}, h, q_{\text{out}}), m, R) \quad (10)$$

Just as an option implements $\omega : \mathcal{I}_\omega \rightarrow \mathcal{T}_\omega$, where $\mathcal{T}_\omega$ is the set of terminal states permitted by β_ω , the contracts define sets of valid inputs $\mathcal{X}_{\text{in}}$ and outputs $\mathcal{X}_{\text{out}}$, and the resulting subagent implements $\text{Subagent}(s) : \mathcal{X}_{\text{in}} \rightarrow \mathcal{X}_{\text{out}}$. The input contract q_{in} plays the role of the initiation set $\mathcal{I}_\omega$, specifying the conditions under which the subagent may be invoked. The instruction file m plays the role of the option policy π_ω , encoding the procedure that carries a valid input to an output. q_{out} plays the role of the termination condition β_ω , constraining what the subagent returns to the main agent. These contracts define a self-contained interface through which information enters and leaves the subagent. We therefore hypothesize that procedural instructions coupled with explicit input-output contracts enable effective subagent execution.

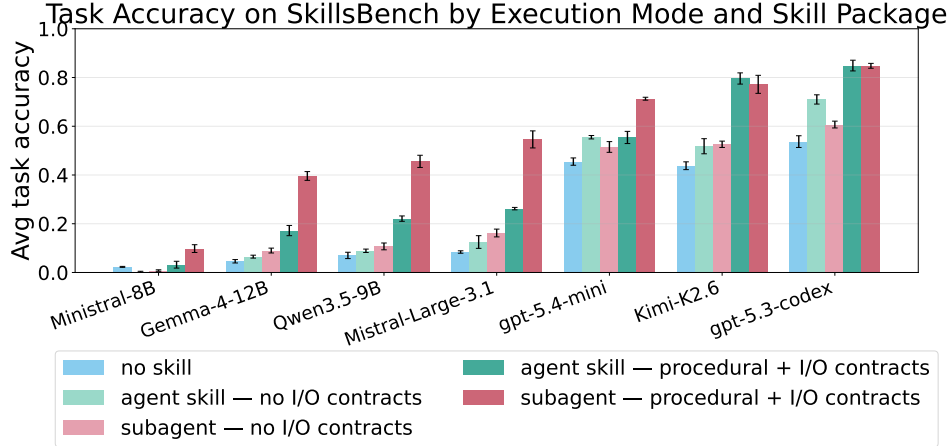


Figure 2: Average task accuracy on SkillsBench for each execution mode (agent skill / subagent) and skill package set, across base LLM models. The *no I/O contracts* packages are the human-authored ones curated with SkillsBench, which describe relevant knowledge but rarely specify a skill’s expected inputs or outputs. The *procedural + I/O contracts* packages are ours, synthesized from successful trajectories to give procedural instructions with explicit input-output contracts.

4 Experiments

We now test the claims made in Section 3. Section 3.2 claims that subagents help only when skill packages expose explicit input-output contracts and pair them with instructions that faithfully deliver on those contracts, and Section 3.1 claims that subagents then trade higher total tokens for lower peak context. We test the contract claim first, comparing both execution strategies on existing curated and our contract-based skill packages. We then increase context pressure by adding distracting tools, testing whether the advantage of subagents grows with context length. Finally, we measure peak context and total token cost directly.

Domain: SkillsBench. SkillsBench [15] is a benchmark specifically designed to evaluate the effectiveness of agent skills, comprising 87 long-horizon agentic tasks across diverse domains. Each task comes with a set of human-authored skill packages, which lets us test agent-skill and subagent execution under a curated set of packages.

We use OpenHands [16] as the agent harness throughout this paper, with limited modifications to the original OpenHands SDK; details can be found at Section A.3.

Synthesizing procedural skill packages with explicit input-output contracts. From inspecting the curated SkillsBench skill packages, we find that very few satisfy the criteria for effective subagent execution discussed in Section 3.2. In particular, most packages describe relevant information but do not clearly specify the inputs expected by the skill or the outputs it should produce.

To construct skill packages better suited for subagent execution, we create a new set of procedural skill packages with explicit input-output contracts for each task. Specifically, we first run OpenHands agents [16] using GPT-5.3 Codex on all SkillsBench tasks for three independent runs, collect successful trajectories, and use Copilot CLI [17], with minimal human intervention, to synthesize skill packages. Using this procedure, we successfully synthesize these well-interfaced procedural skill packages for 64 of the 87 benchmark tasks. More details can be found at Section A.4. The results of this paper are on this 64-task subset of SkillsBench. From now on, we refer to this skill package set as the *synthesized* set.

Main results. Figure 2 shows the main comparison between agent skills and subagents. When using the original curated SkillsBench skills, which lack input-output contracts, agent skills match or outperform subagents across all models. However, the trend reverses when using our procedural skill packages with explicit input-output contracts. On these skill packages, subagents outperform agent skills, with the gain being largest for smaller models that are more bandwidth limited.

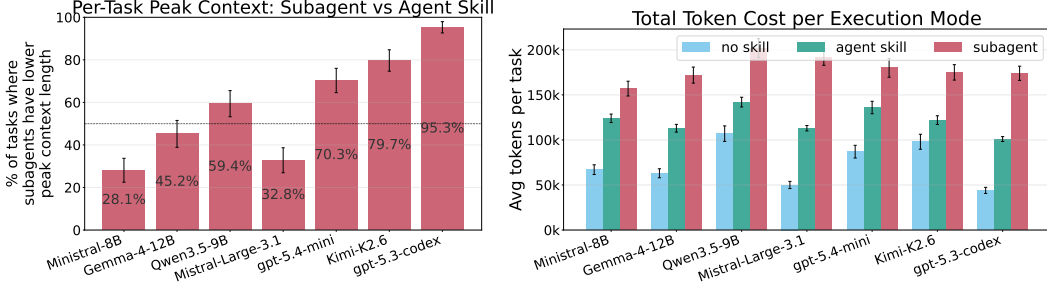


Figure 4: Left: The percentage of tasks whose peak context length used is lower for subagent execution than agent skill execution. Right: Total token used by each execution mode for each LLM model. The skill packages used are from our synthesized set for both left and right figures.

This result supports our central hypothesis: subagent execution becomes beneficial when skills are designed as procedural abstractions with well-defined interfaces. In contrast, when skills primarily contain loosely structured knowledge, directly injecting the skill contents into the main agent’s context, i.e., agent skill execution, is more effective.

Finally, our synthesized skill packages outperform the curated ones across both execution modes. This is not a controlled comparison, since the two sets differ in content, but it suggests that the synthesis procedure produces skills of at least comparable quality to the curated ones.

Scaling with increasing context information.

We next study how both execution strategies scale as input size grows and context pressure intensifies. To do so, we introduce distracting tools whose descriptions are appended to the agent context but are not required for solving the task. This simulates a realistic setting in which agents operate in environments containing many available tools. As the number of distracting tools increases, the initial context becomes substantially longer.

Figure 3 (right) shows that subagent execution degrades more gracefully as the number of distracting tools increases. This confirms the benefit of peak context reduction discussed in Section 3.1: keeping skill instructions out of an already-large context matters even more as that context grows.

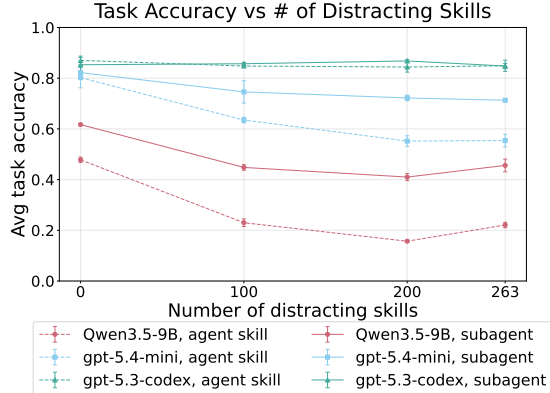


Figure 3: Average task accuracy vs number of distracting skills for each base LLM model and execution mode. The skill packages used are from our synthesized set. Only three LLMs are displayed here for readability, see the full plot at Figure 6.

Peak context length and total token cost. We now test our claim (Section 3.1) that subagents trade communication overhead for reduced peak context.

Figure 4 (left) shows that for the stronger models, such as GPT 5.3 Codex and Kimi K2.6, subagents lower peak context length on over 80% of tasks, as predicted.

For weaker models, the reduction is smaller and sometimes reverses. This appears to reflect early termination under agent-skill execution: as the agent’s reasoning degrades, the run ends without spending enough reasoning effort on the task, so its context stays short. Subagent runs behave differently—each subagent works on a single coherent subtask and can spend the tokens needed to complete it. Peak context length is therefore only comparable across models whose success rates are similar under both execution modes, which the stronger models satisfy and the weaker ones do not.

Figure 4 (right) shows the corresponding token cost. Subagent execution consumes substantially more tokens overall, as expected: an agent skill leaves all information in a single context, while each subagent must be supplied the context it needs, duplicating information the main agent already holds.

5 Related Work

Tool-Calling Language Model Agents. LLMs have become powerful agents when augmented with tools. ReAct interleaves tool use with reasoning through prompting [18], while Toolformer, Gorilla, and ToolLLM instead train models to use tools, fine-tuning them to decide when to call a tool and to generate correct calls [19, 20, 21]. Closely related to our work is HuggingGPT, which uses other LLMs as tools, orchestrated by a central LLM controller [22], similar to how subagents act as LLM-based tools in our setting; unlike our work, however, it does not study skill package execution.

While early tool calling benchmarks evaluated single-turn API routing [23, 24], real-world use increasingly demands long, stateful interactions across many tools and applications [25, 26, 15, 12]. Tasks in SkillsBench, for example, require many tool-calling turns, accumulate substantial information in context, and thus demand methods that scale with context size.

LLM Skill Learning. Skill learning for LLM agents originally focused on programmatic skills. Voyager [27] grows a library of executable code skills for a Minecraft agent. LATM [28], CREATOR [29], and CRAFT [30] instead frame skill acquisition as tool creation, having an LLM author reusable code tools. More recently, SkillCraft [31] learns programmatic skills by composing a sequence of tool calls into skills and benchmarks skill reuse.

Following the wide adoption of skill packages as a skill format [5], we have seen a surge of skill-package learning papers that construct or refine multi-file skill folders directly from agent experience [32, 33, 34]. At their core, these methods mine execution trajectories or failures and use them to synthesize skill packages. These works all execute skill packages as agent skills. Our work also uses skill packages but does not focus on skill-package learning; instead, we show that when skill packages are written or learned to follow a certain design, subagents can be used effectively, and they scale well with context size.

6 Conclusion and Future Work

We studied how reusable skill packages should be executed in long-horizon agentic tasks. While agent skills execute by loading skill instructions into the main agent context, subagents execute skills in separate contexts and return only their outputs. We find that when context overload is an issue, subagent execution performs better and degrades more gracefully than agent skill execution as initial context scales. By distributing computation across multiple context windows, subagents reduce the amount of information that any single context must process.

However, there are limitations to subagent execution. Our experiments suggest that subagents work well only when skill packages encode procedural knowledge with clear input-output contracts. Such skills can be executed largely independently of the main agent’s context, making them suitable for encapsulation in a separate context. Existing curated skills do not always exhibit this property. Nevertheless, we show that procedural, contract-driven skills can be synthesized from successful trajectories, suggesting a practical pathway for learning skills that are suited to subagent execution.

More broadly, our results suggest that scaling agent systems requires reducing context load. Modularity helps control information flow by encapsulating task-specific knowledge behind clear interfaces. From this perspective, subagent execution can be viewed as a mechanism for enforcing modularity.

Looking forward, two directions deserve further investigation. First, while subagent execution reduces peak context length, it incurs additional communication cost, raising the question of how the main agent and subagents should communicate efficiently—designing skills with minimal contracts is one concrete route, but the general problem remains open. Second, we have only begun to explore the problem of skill library organization (Section A.1). Future work could investigate the notion of skill-library refactoring: how should reusable procedures be abstracted, factored, and organized as a skill library grows? In the same way that software engineering studies how large codebases should be structured, scalable agent systems may require principled approaches for organizing procedural knowledge hierarchically.

References

- [1] Edward H. Shortliffe. Computer-based medical consultations: Mycin. *Annals of Internal Medicine*, 85(6):831–831, 1976.
- [2] Douglas B Lenat. Cyc: A large-scale investment in knowledge infrastructure. *Communications of the ACM*, 38(11):33–38, 1995.
- [3] Ronald J Brachman, Hector J Levesque, and Maurice Pagnucco. *Knowledge representation and reasoning*, volume 1. Elsevier, 2004.
- [4] Robyn Speer, Joshua Chin, and Catherine Havasi. Conceptnet 5.5: An open multilingual graph of general knowledge. In *Proceedings of the AAAI conference on artificial intelligence*, volume 31, 2017.
- [5] Anthropic. Agent Skills. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>, 2025. Accessed: 2026-08-08.
- [6] Richard S Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2):181–211, 1999.
- [7] Ronald Parr and Stuart Russell. Reinforcement learning with hierarchies of machines. *Advances in neural information processing systems*, 10, 1997.
- [8] Thomas G Dietterich. Hierarchical reinforcement learning with the maxq value function decomposition. *Journal of artificial intelligence research*, 13:227–303, 2000.
- [9] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.
- [10] Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Bodapati, Aram Galstyan, Azton Wells, Roy Schwartz, Eliu A Huerta, and Hao Peng. Context length alone hurts llm performance despite perfect retrieval. *arXiv preprint arXiv:2510.05381*, 2025.
- [11] Eunjin Hong, Sumin Cho, and Juae Kim. Exploring working memory capacity in llms: From stressors to human-inspired strategies. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 1727–1744, 2025.
- [12] Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, Xiaochen Wang, Rui Ge, Yuxuan Cao, Yuzhen Huang, Wei Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. In *International Conference on Learning Representations*, volume 2026, pages 53359–53394, 2026.
- [13] Anthropic. How and when to use subagents in Claude Code, 2026. Accessed: 2026-09-04.
- [14] Tobias Schnabel, Kiran Tomlinson, Adith Swaminathan, and Jennifer Neville. Lost in transmission: When and why llms fail to reason globally. *Advances in Neural Information Processing Systems*, 38:138944–138982, 2026.
- [15] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026.
- [16] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919, 2025.
- [17] GitHub. GitHub Copilot CLI. <https://github.com/github/copilot-cli>, 2026. Accessed: 2026-08-31.
- [18] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [19] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.

- [20] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024.
- [21] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, volume 2024, pages 9695–9717, 2024.
- [22] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36:38154–38180, 2023.
- [23] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 3102–3116, 2023.
- [24] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- [25] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.
- [26] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment, 2025.
- [27] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [28] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. In *International Conference on Learning Representations*, volume 2024, pages 54067–54089, 2024.
- [29] Cheng Qian, Chi Han, Yi Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. Creator: Tool creation for disentangling abstract and concrete reasoning of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 6922–6939, 2023.
- [30] Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi Fung, Hao Peng, and Heng Ji. Craft: Customizing llms by creating and retrieving from specialized toolsets. In *International Conference on Learning Representations*, volume 2024, pages 40097–40125, 2024.
- [31] Shiqi Chen, Jingze Gai, Ruochen Zhou, Jinghan Zhang, Tongyao Zhu, Junlong Li, Kangrui Wang, Zihan Wang, Zhengyu Chen, Klara Kaleb, et al. Skillcraft: Can llm agents learn to use tools skillfully? *arXiv preprint arXiv:2603.00718*, 2026.
- [32] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026.
- [33] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, et al. Coevoskills: Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*, 2026.
- [34] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjin Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.
- [35] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.
- [36] Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, et al. Skillx: Automatically constructing skill knowledge bases for agents. *arXiv preprint arXiv:2604.04804*, 2026.
- [37] Yuchen Liu, Junhao Hu, Yingdi Shan, Ge Li, Yanzhen Zou, Yihong Dong, and Tao Xie. Llmi-grate: Transforming "lazy" large language models into efficient source code migrators. *arXiv preprint arXiv:2503.23791*, 2025.

A Appendix

A.1 Organizing Libraries of Skills to Decrease Initial Context Load

We next ask whether hierarchically organizing a skill library can amplify the benefits of context isolation. Recent agent systems increasingly lazy-load MCP servers, exposing capabilities only when needed, which keeps the initial context short and lowers peak context length in later turns. We test an analogous strategy for skill libraries: rather than exposing every skill at once, we organize skills into a hierarchy and expose only a subset at each level.

Library organization. When a library contains many skills, exposing all of them to the agent at once can be overwhelming for the agent. A hierarchical library instead exposes only a small set of top-level nodes at first; selecting a node either performs a task (if it is a leaf skill) or reveals a further set of child nodes to choose from (if it is a *routing* node). The agent navigates the hierarchy step by step rather than seeing the whole library up front. We compare four different library organizations, all with the leaf nodes being the well-interfaced procedural skills evaluated in Figure 2:

- **Flat library.** No hierarchy: all leaf skills are exposed to the agent simultaneously. This is the baseline used throughout Figure 2.
- **Hierarchical tree library.** An LLM groups the leaf skills into a tree of routing nodes, where each routing node has exactly one parent, so there is a single path from the root to any leaf.
- **Hierarchical graph library.** The same LLM-built organization as the tree library, except a node may have multiple parents, so the structure is a DAG rather than a tree and a leaf can be reached via more than one path.
- **Two-level task-tree library.** A simpler baseline (no LLM involved): leaf skills are grouped by the task they belong to, giving a fixed two-level hierarchy of task node to leaf skills.

Execution modes. Independently of library structure, we vary how nodes are executed once selected:

- **Fully agent skill:** every node, routing or leaf, runs inline in the main agent’s context.
- **Fully subagent:** every node runs as an isolated subagent call.
- **Hybrid:** routing (higher-level) nodes run as agent skills, while leaf skills run as subagents.

Results. Figure 5 shows that hierarchical organization gives additional gains on top of our already well-interfaced procedural skill packages. The task-tree and hierarchical-graph libraries perform best, and the LLM-built graph outperforms the LLM-built tree, suggesting that allowing multiple paths to a leaf skill helps. We did not observe further gains from deeper hierarchies in our experiments, though we expect benefits from deeper structure to emerge with libraries of thousands of skills.

Across all four library types, the hybrid execution mode is consistently best: routing nodes as agent skills, leaf skills as subagents. This mirrors lazy-loading in MCP-based harnesses and supports our broader *effective subagent hypothesis*: routing nodes carry no procedural knowledge or clear input-output contract, so they are better executed inline as agent skills, while leaf skills – which do have such contracts – benefit from running as isolated subagents. We leave a systematic study of deeper hierarchies and tree-vs-graph structure to future work.

A.2 Additional results on SkillsBench

Here, we include the average task accuracy vs number of distracting skills plot, for Section 4, with more base models. The interpretation of the results remains the same; we choose to only include only three LLMs in the main text for better readability.

We also report number of skill calls per task at Figure 7. We see that subagent execution leads to more skill calls per task across all base LLMs. With agent-skill execution, we think that the LLMs start to underthink as their context grows [37], leading to lower skill calls on average.

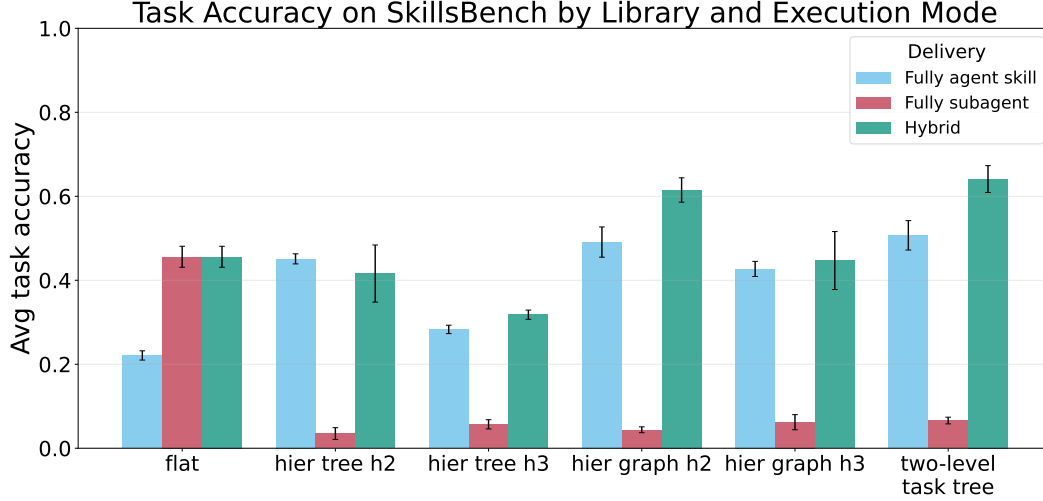


Figure 5: Average task accuracy on SkillsBench for each library organization (flat, hierarchical tree, hierarchical graph, and two-level task tree) and execution mode (fully agent skill / fully subagent / hybrid) on Qwen3.5-9B. *h2* and *h3* means height=2 and height=3 respectively.

A.3 OpenHands Agent Implementation Details

We use OpenHands [16] as one of the underlying agent frameworks in our experiments. When running each SkillsBench task, we spin up a docker container and run OpenHands agent inside the task’s Docker container as the benchmark harness’s registered agent. We route every LLM call through a proxy that we control, which lets us inject system prompts, apply sampling overrides, and log every request/response pair uniformly.

Prompt templates. We replace the Openhands’ own vanilla system prompt with a much simpler prompt shown at Figure 8. When subagents get spawned, they are initialized with the prompt shown at Figure 9.

Other modifications to vanilla OpenHands SDK We also implement two modifications to the vanilla OpenHand SDK. First is we extend the subagent timeout. The OpenHands SDK hardcodes a 5-min timeout on every MCP tool call. Since our subagents are treated as MCP tools, we extend this timeout to two hours. Second is we remove OpenHands’ native skills. We also disable OpenHands’ native auto-skill retrieval.

A.4 Well-interfaced Procedural Skill Authoring details

As mentioned in Section 4, we mainly use Copilot CLI, powered by Claude Opus 5, to come up with our well-interfaced procedural skill packages. Initially, we ask Copilot CLI to come up with a set of 3-5 of such skill packages for a single task, called edit-pdf. Then, we manually intervene to ensure the the synthesized skills do have an "expected input" and "output" description, as part of their skill description, and the skill instructions faithfully try to transform the input to output. Then, we ask Copilot CLI to spawn subagents to do write a set of 3-5 well-interfaced procedural skill packages for each of the task with successful trajectories, using the skill set from the initial task, edit-pdf, as an example. Copilot CLI uses GPT-5.5 to power the subagents.

We show an example output skill descriptions from this process at Figures 10 and 11.

A.5 Computational Resources

Experiments were run on NVIDIA A100 GPU nodes. Open-weight models (the Qwen3.5 family at 2B/4B/9B parameters, Ministral-3-8B, and Gemma-4-12B-it) were served locally with vLLM directly on an A100 node’s GPU. Closed-weight or large models(the GPT-5.x family, Mistral-Large-3, Kimi-K2.6) were accessed through a hosting API and requires no GPU itself.

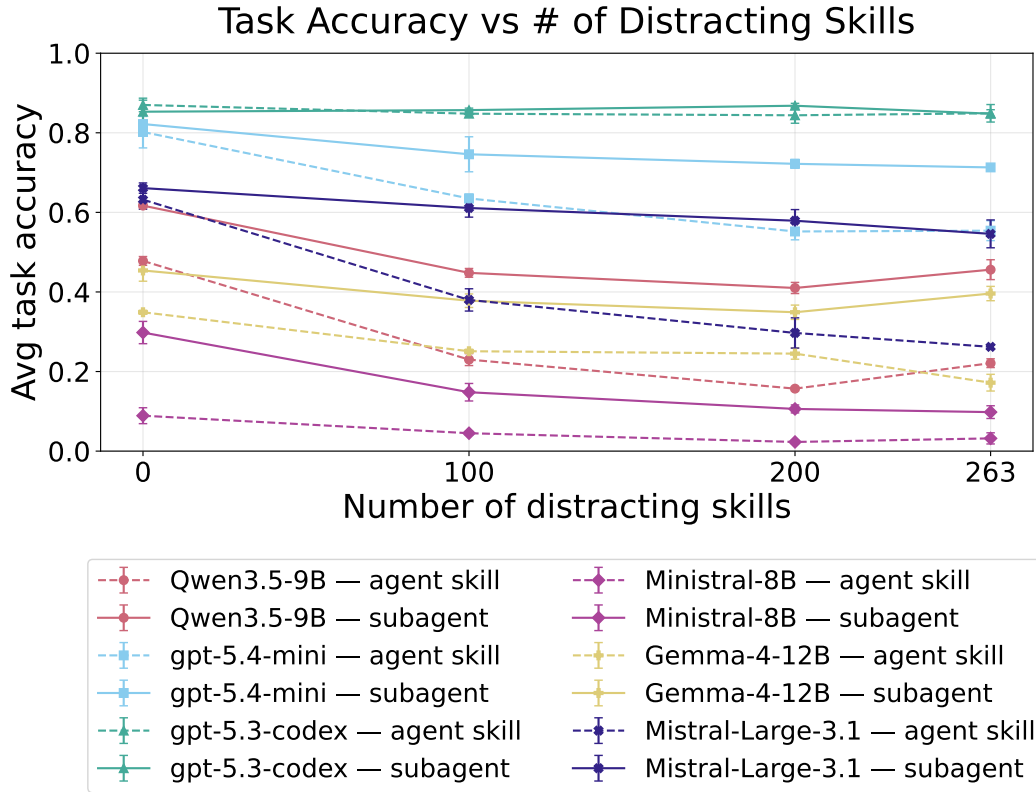


Figure 6: Average task accuracy vs number of distracting skills for each base LLM model and execution mode. The skill packages used are from our synthesized set.

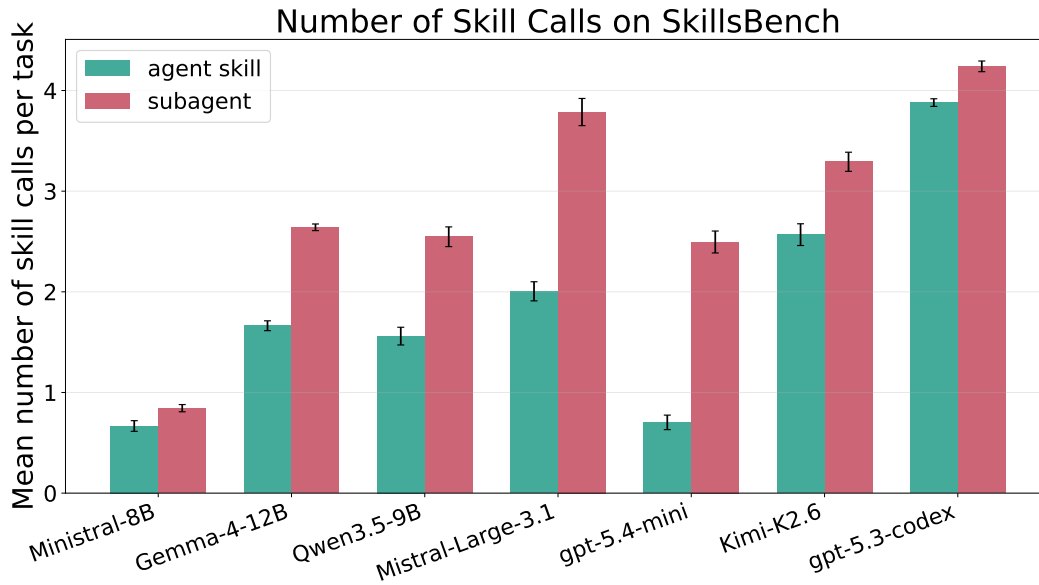


Figure 7: Number of skill calls for each base LLM model. The skill packages used are from our synthesized set.

Default system prompt (simple)

You are AI agents with access to tools. Please solve the user-specified task as best as you can. Please always use tools whenever they are relevant -- don't try to solve subtasks on your own. Please try to use the <tool1>, <tool2>, ... when possible.

Figure 8: The system prompt used for our OpenHands agent. *tool1*, *tool2*, and so on list the name of the skills (agent skills / subagents) available.

Subagent task template

You will be given a task and the knowledge on how to solve it

TASK:
{task}

INPUT INFORMATION: [only when input_info is non-empty]
{input_info}

KNOWLEDGE:
{skill_body}

Resources mentioned in KNOWLEDGE are available at: {skill_dir}

IMPORTANT: When you are done, return the output this skill produces in your FINAL message. If the output is a file (or files) you created or edited, state its exact path in your final message; otherwise put the actual result text in your final message. Do not end without reporting the output.

Figure 9: Initial prompt for a spawned subagent. *skill_body* is the content in a SKILL.md file. *task* and *input_info* are given to the subagent by the main agent.

Skill vulnerability-record-normalization description

Convert scanner JSON into normalized CSV-ready vulnerability records using the task severity filter and deterministic CVSS/fixed-version fallbacks.

Expected input:

- the scanner JSON report path from the scan step
- the required severity filter and CSV columns from the audit plan

Output:

- a JSON list of normalized records with Package, Version, CVE_ID, Severity, CVSS_Score, Fixed_Version, Title, and Url fields.

Figure 10: A description of a well-interfaced procedural skill, vulnerability-record-normalization

Skill offline-vulnerability-scan description

Run a reproducible vulnerability scan for a dependency lockfile and save the full scanner result as JSON for later normalization.

Expected input:

- the audit plan containing the dependency lockfile path and desired JSON report path
- the scanner/cache strategy, including any local Trivy cache directory if available

Output:

- a JSON vulnerability report saved on disk plus a summary of total findings and counts by severity.

Figure 11: A description of a well-interfaced procedural skill, offline-vulnerability-scan